UNIVERSIDADE FEDERAL DA PARAÍBA

CENTRO DE INFORMÁTICA

RED BALLOON

DOCUMENTO DE REQUISITOS

Davi de Lacerda Teixeira	20230012251
Herick José de Freitas	20230021813
João Victor F. da Silveira	20230012298
Josué Guedes Ferreira	20230012313
Lael Gustavo B. R. de Lima	20230021715
Luis Gustavo L. Reis	20230089650

João Pessoa

2025

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
11/12/2025	1.00	Primeira versão do documento de requisitos, que traz a definição do sistema proposto para desenvolvimento, junto com suas funcionalidades básicas, público-alvo, entidades de classe, técnicas de elicitação de requisitos e listagem de requisitos funcionais, não funcionais e afins.	Davi de L. Teixeira Herick José de Freitas João Victor F. da Silveira Josué Guedes Ferreira Lael G. B. R. de Lima Luis Gustavo L. Reis
26/02/2026	1.50	Correção da primeira versão do documento, adicionado descrições detalhadas da elicitação e artefatos gerados, bem como melhorando especificações de requisitos não funcionais.	Davi de L. Teixeira João Victor F. da Silveira
04/03/2026	2.00	Adição de descrição dos três principais casos usos do sistema, bem como suas especificidades.	Davi de L. Teixeira Herick José de Freitas João Victor F. da Silveira
05/03/2026	2.50	Adição de diagrama para todos os casos de uso documentados e atualização de diagrama de classes de análise.	Josué Guedes Ferreira Lael G. B. R. de Lima Luis Gustavo L. Reis

1. INTRODUÇÃO

1.1. Propósito do documento

Este documento propõe definir e detalhar as especificidades do sistema **Red Balloon**, uma plataforma que busca reunir usuários para resolução compartilhada de problemas de programação. Nesse contexto, além de definir escopo, público-alvo e funcionalidades esperadas para o projeto, o documento apresenta um glossário que define as principais entidades usadas no sistema, bem como os mecanismos de elicitação de requisitos, que permitiram a listagem de requisitos funcionais e não-funcionais, bem como de desempenho, usabilidade e segurança.

1.2. Visão geral do documento

Esta introdução fornece as informações necessárias para fazer um bom uso deste documento, explicitando seus objetivos e as convenções que foram adotadas no texto, além de conter uma lista de referências para outros documentos relacionados. As demais seções apresentam a especificação do sistema **Red Balloon** e estão organizadas como descrito abaixo.

- Seção 2 – Descrição geral do sistema: apresenta uma visão geral do sistema, caracterizando qual é o seu escopo e descrevendo seus usuários.
- Seção 3 – Requisitos funcionais (casos de uso): especifica todos os requisitos funcionais do sistema, descrevendo os fluxos de eventos, prioridades, atores, entradas e saídas de cada caso de uso a ser implementado.
- Seção 4 – Requisitos não funcionais: especifica todos os requisitos não funcionais do sistema, divididos em requisitos de usabilidade, confiabilidade, desempenho, segurança, distribuição, adequação a padrões e requisitos de hardware e software.
- Seção 5 **–** Descrição da interface com o usuário: apresenta desenhos, figuras ou rascunhos de telas do sistema.

1.3. Documentos relacionados

Documentos relacionados ao **Red Balloon** e/ou mencionados nas seções a seguir:

1. Diagrama de Entidade-Relacionamento do Sistema; 15 de dezembro, 2025; Equipe de desenvolvimento backend; github.com/sunny-fellow/RedBalloon/blob/main/docs/er_diagram.png.

2. DESCRIÇÃO GERAL

O sistema proposto, chamado **Red Balloon**, constitui-se de um ambiente web interconectado cujo objetivo é fornecer um espaço no qual usuários possam criar problemas de programação e resolvê-los de maneira colaborativa e competitiva com outras pessoas. Nesse sentido, a aplicação surge com o propósito de oferecer um ambiente comunitário para entusiastas da computação, fomentando o compartilhamento de código e a competição saudável entre eles. Essa proposta se destaca por permitir que usuários comuns proponham problemas de código e discutem sua validade por meio da interação entre eles e a proposição de soluções por cada membro interessado na questão.

2.1. Motivação

O projeto é proposto à medida que se percebe que muitas plataformas de resolução de código restringem a funcionalidade de criação e proposição de problemas e soluções, limitando o usuário apenas à criação de sua própria solução para uma questão existente e, consequentemente, individualizando o processo de criação e aprendizado. O **Red Balloon** busca corrigir essa limitação ao permitir que qualquer usuário proponha novos problemas e discuta com outros membros possibilidades de solução, além de permitir que diferentes usuários disputem por resoluções melhores. Desse modo, é criado um sistema integrado marcado pela interação entre membros, compartilhamento do aprendizado, competição saudável e colaboração constante.

2.2. Problemas identificados

As plataformas atuais para interação entre programadores tem um foco apenas na discussão de problemas pontuais ou no trabalho lúdico de problemas com complexidade limitada em seu escopo e contexto. Foi identificada uma ausência de um ambiente virtual onde programadores pudessem não apenas aprender mais sobre suas linguagens de escolha e resolver dúvidas pequenas, mas também compartilhar problemas de programação em seu contexto completo e ter discussões a fundo quanto às particularidades de como encontrar a solução do problema e a melhor maneira de implementá-la.

Também foi observado que os ambientes para aprendizado de programação na internet tendem a especializar em linguagens fixas nas instâncias em que procuram ter mais complexidade nas discussões, algo que ocorre dadas as limitações dos próprios desenvolvedores da plataforma terem de suprir os materiais com problemas de código para os usuários. Se observa que não há opção popular de plataforma com problemas feitos do público para o público.

2.3. Visão geral do sistema

A principal funcionalidade proposta para o **Red Balloon** é a criação e submissão de questões de código na plataforma. Nesse caso, o usuário cria um problema de programação e especifica a linguagem adequada e um arquivo com as soluções daquele problema, caso ele as possua, ou um algoritmo de verificação de soluções. Caso não haja nenhum critério de solução a priori, o problema pode subir no sistema sem uma resolução específica, permitindo que outros usuários possam buscá-la por conta própria. Essa interação representa mais uma funcionalidade da aplicação: comunicação por meio de um método de fóruns. Para cada problema submetido, haverá um espaço de comunicação no qual outros usuários podem interagir e comentar acerca da questão, discutindo melhores estratégias para sua resolução e, se necessário, anexando suas próprias versões de código resolvedor.

Além do escopo de discussão e aprendizado colaborativo, o projeto também contempla a criação de salas de programação competitiva, nas quais os usuários podem selecionar um conjunto de problemas já cadastrados no sistema e convidar outros participantes para disputas de resolução. Nessas salas, os usuários conectados simultaneamente devem propor soluções para os desafios apresentados, sendo recompensados com pontuações maiores conforme a rapidez na resposta ou a eficiência da solução, considerando critérios como tempo de execução. Para utilizar qualquer funcionalidade da plataforma, cada indivíduo deverá criar uma conta.

2.4. Usuários do sistema

O sistema **Red Balloon** tem como público-alvo estudantes e entusiastas da área de computação, especialmente aqueles interessados em programação e desenvolvimento de software. Esses usuários frequentemente buscam um ambiente unificado onde possam praticar resolução de problemas, compartilhar código e receber feedback qualificado, mas enfrentam desafios como a falta de plataformas que integrem discussão colaborativa, análise de soluções e espaços de competição. Além disso, muitos têm dificuldade em encontrar oportunidades para comparar abordagens, otimizar algoritmos e interagir com outros programadores em um contexto seguro e moderado. O **Red Balloon** foi projetado para atender a essas necessidades, oferecendo um ambiente integrado que combina compartilhamento de código, fórum técnico e salas de programação competitiva, permitindo aos usuários aprimorarem suas habilidades, colaborarem entre si e participarem de disputas que incentivam desempenho e eficiência.

2.5. Suposições e restrições gerais

O sistema **Red Balloon** deve contar com algumas suposições básicas de desenvolvimento, que estabelecem limitações sobre o processo de desenvolvimento. Além disso, o sistema está sujeito a um conjunto de fatores que limitam os requisitos estabelecidos à frente nesse documento. Dentre elas, pode-se destacar:

- A organização e divisão de trabalho da equipe deve ser feita seguindo os princípios do desenvolvimento ágil, a partir da aplicação do método Scrum.
- A equipe de desenvolvimento deve verificar, após cada sprint, o cumprimento de requisitos básicos de segurança e robustez do sistema por meio de testes.
- O sistema de API desenvolvido para o projeto deve encapsular as funcionalidades em módulos que funcionem como serviços.
- As funções devem estar devidamente documentadas, e aquelas que recebem dados de entrada devem utilizar *models* para validação.
- Os serviços e funcionalidades do sistema devem ser implementados na linguagem Python e distribuído via web por meio da framework Flask.
- O sistema desenvolvido deve respeitar a atual legislação de proteção de dados para garantir a segurança dos usuários.
- A funcionalidade de competições em salas interativas exige o uso de mecanismos de comunicação persistentes, como WebSockets.
- A estrutura do banco de dados para o sistema deve seguir a 3º forma normal.

Outras questões importantes não definidas neste tópico foram demarcadas como requisitos não-funcionais, vide seção 5.2.

3. GLOSSÁRIO

3.1. Conceitos gerais

Nome	Descrição
Balloons	Refere-se às curtidas que os usuários podem deixar em problemas, soluções ou comentários. Nessas entidades, trata-se de um número inteiro, representando a quantidade total de curtidas.
Comunidade	Refere-se ao conjunto de jogadores que interagem com problemas, soluções e comentários, além de participar das salas.
Chat	Refere-se ao canal de comunicação em uma sala, o qual os jogadores poderão se utilizar para discutir questões envolvendo os desafios.
Mensagem	Refere-se a uma string enviada de um jogador para ser disposta no *chat* de uma sala.
Ranking	Refere-se à classificação parcial de jogadores associados a uma determinada sala competitiva, de acordo com o tempo de execução da solução enviada pelo jogador.

3.2. Entidades

3.2.1. Perfil de Jogador

Entidade que representa as informações básicas do cadastro realizado pelo usuário do site.

Atributo	Tipo
Nome	String
E-mail	String
Login	String
Senha	String
Imagem	Arquivo em formato PNG ou JPEG

3.2.2. Sala

Entidade que representa as informações básicas de uma sala criada por jogadores do site.

Atributo	Tipo
Código	Inteiro
Quantidade máxima	Inteiro (*small int*)
Quantidade de usuários	Inteiro (*small int*)
Privada ou pública	Valor booleano
Senha	NULL ou string
Estado de aceitação de submissões	Bool
Chat habilitado	Bool

3.2.3. Problema

Entidade que representa as informações básicas de um problema gerado por um jogador.

Atributo	Tipo
Descrição	String
Dificuldade	Enum
Verificado	Booleano
Balloons	Inteiro
Limite de tempo	Inteiro
Limite de memória	Inteiro
Modo de validação	Enum
Algoritmo de checagem	Arquivo

3.2.4. Solução

Entidade que representa as informações básicas de uma solução gerada por um usuário em uma sala ou em um problema.

Atributo	Tipo
Título	NULL ou string
Conteúdo	String
Comentário	String
Balloons	Inteiro
Tempo gasto	Float
Memória gasta	Float

3.2.5. Comentário

Entidade que representa um canal de troca de mensagens gerado simultaneamente com uma sala.

Atributo	Tipo
Conteúdo	String
Balloons	Inteiro

4. ELICITAÇÃO DE REQUISITOS

A elicitação de requisitos é uma etapa fundamental no processo de definição e entendimento das necessidades que orientarão o desenvolvimento do sistema. Nessa fase, diversas técnicas podem ser aplicadas, de forma combinada ou isolada, para garantir uma visão ampla, precisa e alinhada às expectativas dos stakeholders. Para o presente projeto, foram adotadas duas técnicas principais de elicitação: **brainstorming** e **análise de sistemas similares**. A seguir, detalham-se os procedimentos, justificativas e informações sobre a condução de cada técnica aplicada.

4.1. Brainstorming

- **Data:** 09/12/2025
- **Horário:** 08h00min
- **Local:** João Pessoa – PB
- **Participantes:**
    a. Davi de Lacerda Teixeira
    b. Herick José de Freitas
    c. João Victor Fernandes da Silveira
    d. Josué Guedes Ferreira
    e. Lael Gustavo Batista Ribeiro de Lima
    f. Luis Gustavo Lacerda Reis

O brainstorming foi escolhido por ser uma técnica eficaz para a geração de ideias e identificação de requisitos iniciais, especialmente em projetos novos ou com escopo ainda em construção. A dinâmica teve caráter colaborativo e descontraído, possibilitando que todos os participantes contribuíssem livremente, sem julgamento prévio das sugestões.

Durante a sessão, os participantes foram incentivados a imaginar o funcionamento ideal da plataforma, considerando tanto a perspectiva de um usuário comum quanto de um criador de desafios. A discussão foi guiada por perguntas como: *"O que é indispensável para que o sistema funcione?"*, *"Como os usuários irão interagir entre si?"* e *"Como as competições ocorrerão?"*. A partir dessas discussões, foram levantados e posteriormente consolidados os seguintes requisitos funcionais:

- **RF01**: surgiu da necessidade básica de identificação e personalização da experiência na plataforma.
- **RF02**: derivado da necessidade de autenticação segura para acesso às funcionalidades restritas.

- **RF04**: emergiu da ideia central de promover interação em tempo real entre usuários.
- **RF06**: resultado da proposta de incorporar elementos sociais semelhantes a fóruns.
- **RF07**: originado da proposta de tornar a plataforma colaborativa, permitindo que membros contribuam com novos desafios.
- **RF08**: decorrente da intenção de estimular reconhecimento e reputação dentro da comunidade.
- **RF10**: identificado como elemento essencial para promover colaboração durante resoluções conjuntas ou contests.
- **RF11**: necessidade de controle organizacional, como definir regras, gerenciar participantes e configurar atributos da sala.
- **RF12**: proposto como mecanismo de gamificação para aumentar o engajamento.

Os requisitos levantados nesta etapa foram inicialmente registrados de forma bruta e posteriormente refinados, eliminando ambiguidades e organizando-os por categoria e prioridade. Durante o processo de elicitação, foi criado um documento de apoio para listagem e visualização dos tópicos que eram propostos durante o *brainstorming*, que resultou na pontuação de aspectos técnicos que o sistema deve ter e os requisitos brutos obtidos. Tal documento pode ser encontrado no seguinte link: https://docs.google.com/document/d/1sc0Mmr3gqG2NOpDIRXdV67svOk2O5p3kJz5pmt2pCgs/edit?usp=sharing.

4.2. Análise de Sistemas Similares

- **Data:** 11/12/2025
- **Horário:** 08h00min
- **Local:** João Pessoa – PB
- **Participantes:**
    a. Davi de Lacerda Teixeira
    b. Herick José de Freitas
    c. João Victor Fernandes da Silveira
    d. Josué Guedes Ferreira
    e. Lael Gustavo Batista Ribeiro de Lima
    f. Luis Gustavo Lacerda Reis

A análise de sistemas similares consistiu na observação de plataformas consolidadas no mercado, como o BeeCrowd, o Codeforces e o LeetCode, com o objetivo de identificar boas práticas, limitações e oportunidades de inovação. Foram analisados principalmente os seguintes aspectos:

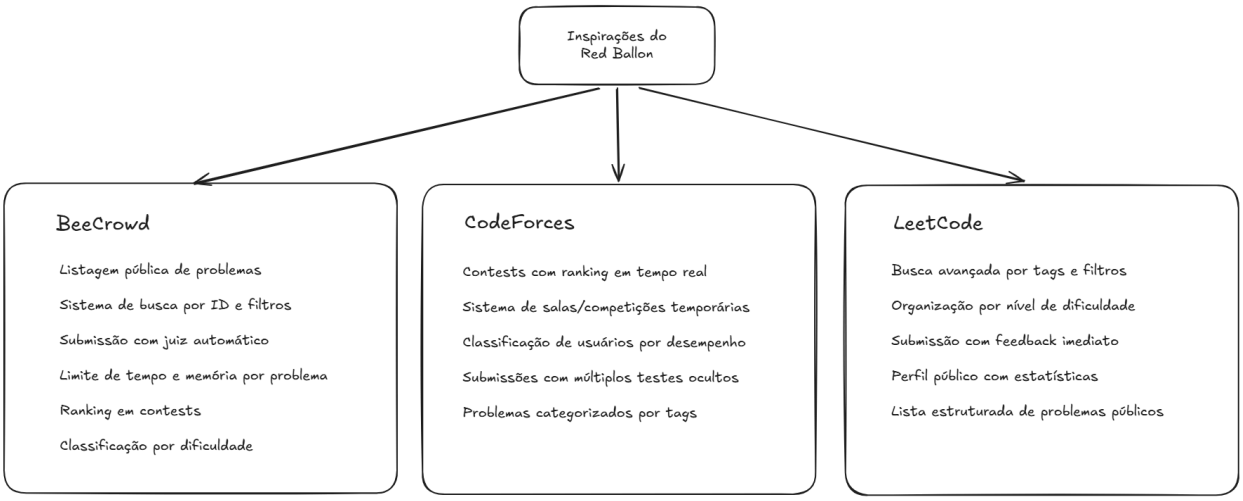
- navegabilidade e experiência do usuário (UX/UI);

- organização e categorização de desafios;
- mecanismos de submissão e validação automática de soluções;
- estruturação de competições e rankings;
- sistemas de busca e filtragem de problemas.

A partir dessa investigação comparativa, foi possível consolidar e detalhar requisitos que complementam os resultados do brainstorming, especialmente no que se refere à organização e funcionamento técnico da plataforma. Dessa etapa derivaram principalmente:

- **RF03**: inspirado na estrutura de listagem clara e organizada presente nas plataformas.
- **RF05**: identificado como funcionalidade essencial para escalabilidade e usabilidade em sistemas com grande volume de desafios.
- **RF09**: fundamentado na observação dos sistemas de juízes online e da necessidade de avaliação automatizada das soluções.
- **RF12**: refinado a partir da análise dos modelos de ranking e classificação competitiva utilizados nas plataformas estudadas.

Diferentemente do brainstorming, que teve caráter mais criativo e exploratório, a análise de sistemas similares permitiu validar ideias previamente levantadas e transformá-las em requisitos mais concretos, alinhados às práticas consolidadas no mercado. Além disso, essa técnica contribuiu para identificar padrões funcionais considerados essenciais pelos usuários, reduzindo riscos de omissão de funcionalidades críticas. O gráfico de funcionalidades analisadas de cada aplicação, artefato produzido durante a análise, pode ser encontrado abaixo:



4.3. Considerações Finais

As técnicas aplicadas se complementam: o brainstorming permitiu a construção de um panorama inicial amplo, enquanto a análise de sistemas similares forneceu referências práticas para validação e aprimoramento das ideias. O conjunto resultante de requisitos obtidos nesta fase servirá de base para as próximas etapas do processo de desenvolvimento, como análise, modelagem e priorização.

5. ANÁLISE DE REQUISITOS

5.1. Requisitos funcionais

As prioridades serão divididas em: opcional, média, alta, essencial. Apenas as essenciais são consideradas para MVP e, nessa tabela, estão marcadas em **negrito**.

Cód.	Descrição	UC Relacionado	Prioridade
RF01	O usuário deve ser capaz de se cadastrar no sistema	Autenticação	Alta
RF02	O usuário deve ser capaz de realizar o login de conta, através de (login ou email) e senha	Autenticação	Alta
RF03	O site deve exibir os problemas públicos criados na plataforma	Acesso a Problemas	**Essencial**
RF04	O usuário deve ser capaz de adicionar e participar de salas	Participação de salas	**Essencial**
RF05	O site deve permitir a busca por problemas específicos através de título e filtros, como tags (exemplo: *#bitmask*)	Acesso a problemas	Média
RF06	O usuário deve ser capaz de realizar comentário em cima de problemas e soluções	Interação e Comunidade	Alta
RF07	Os usuários devem ser capaz de criar problemas	Acesso a problemas	**Essencial**
RF08	Os usuários devem poder visualizar os perfis de outros	Interação e Comunidade	Média
RF09	Os usuários devem ser capazes de submeter soluções, sendo possível checar a própria resposta posteriormente em seu perfil, de problemas individualmente ou em salas	Submissão de Soluções	**Essencial**
RF10	As salas devem conter um chat	Participação de salas	Alto
RF11	O usuário que criou uma sala deve ser capaz de gerenciar atributos da sala e dos participantes	Participação de salas	Alto
RF12	As soluções submetidas em uma sala terão uma pontuação associada para a exibição posterior de um rank durante contests	Participação de salas	**Essencial**
RF13	O usuário deve poder, fora de sala, visualizar soluções aceitas de problemas.	Acesso a problemas	Médio

5.2. Requisitos não funcionais

Cód.	Descrição	Prioridade	Categoria
RNF01	O sistema deve exigir autenticação obrigatória para a participação de contests	Alta	Segurança
RNF02	As senhas devem ser armazenadas utilizando hash criptográfico seguro	Média	Segurança
RNF03	O sistema deve utilizar HTTPS para autenticação e limitar a no máximo 5 tentativas consecutivas de login malsucedidas antes de bloqueio temporário da conta.	Média	Segurança
RNF04	O sistema deve rejeitar submissões que excedam 2 segundos de tempo de execução ou 256 MB de memória, retornando veredito apropriado ao usuário	Essencial	Desempenho
RNF05	O sistema deve manter a sessão do usuário ativa durante contests por no máximo 2 horas sem necessidade de novo login	Essencial	Usabilidade
RNF06	O sistema deve manter até 10 jogadores em uma sala.	Essencial	Desempenho
RNF07	O sistema deve garantir isolamento entre sessões e execuções de usuários durante contests.	Alta	Segurança
RNF08	As soluções devem poder ser submetidas nas linguagens: C, C++, Python e Java	Essencial	Usabilidade
RNF09	O sistema deve utilizar o mesmo conjunto de casos de teste para todas as linguagens e aplicar limite de 2 segundos para linguagens compiladas e 3 segundos para linguagens interpretadas, garantindo avaliação justa entre os participantes.	Alta	Confiabilidade
RNF10	O site deve funcionar nos navegadores: Chrome, Microsoft Edge, Firefox e Safari	Média	Usabilidade
RNF11	O sistema deve manter layout funcional e legível nas resoluções 1366x768 e 1920x1080, sem sobreposição de elementos ou perda de conteúdo.	Média	Usabilidade
RNF12	A interface deve permitir que pelo menos 80% dos usuários realizem ações básicas após no máximo 5 minutos de uso inicial.	Média	Usabilidade
RNF13	Todos os campos de entrada devem possuir limites mínimos e máximos definidos, como textos com até 255 caracteres e limite de tempo de execução entre 1 e 10 segundos.	Alta	Segurança

RNF14	O sistema deve implementar validação e sanitização de entradas para prevenir ataques como SQL Injection e manipulação de parâmetros via URL.	Alta	Segurança
RNF15	O sistema deve suportar a criação e manutenção de no máximo 10.000 salas simultâneas ativas.	Essencial	Desempenho
RNF16	O sistema deve remover automaticamente salas que permaneçam sem participantes por mais de 5 minutos.	Média	Confiabilidade

6. ESPECIFICAÇÃO DE REQUISITOS

[UC001] Criação de problemas de programação

Descrição: em sua conta pessoal, o usuário pode desenvolver um problema de programação e publicá-lo no sistema para que outros usuários possam resolvê-los.

Ator: Usuário.

Prioridade	**X**	Essencial		Importante		Desejável

Entradas e pré-condições:

1. Pré-condições:
    - O usuário deve estar autenticado no sistema;
    - O usuário deve ter acesso à interface de criação de problemas.
2. Entradas obrigatórias:
    - Descrição do problema.
3. Entradas opcionais:
    - Dificuldade do problema;
    - Limite de tempo;
    - Limite de memória;
    - Algoritmo de checagem.

Saídas e pós-condições:

1. Saídas:
    - Confirmação de publicação exibida ao usuário;
    - Página própria do problema;
    - Em caso de erro, mensagem de erro detalhando o problema.
2. Pós-condições:
    - O problema é armazenado no banco de dados do sistema (caso seja bem-sucedido).
    - O problema fica disponível para visualização e resolução por outros usuários.
    - Se houver falha na publicação, o problema não é armazenado, mantendo consistência.
    - Usuários podem curtir o problema para torná-lo mais relevante.

Fluxo de eventos principal

1. O usuário acessa sua conta pessoal no sistema.
2. O usuário seleciona a opção “Criar Problema” presente na página inicial.
3. O sistema exibe a página de criação de problema.
4. O usuário preenche o campo obrigatório da descrição.
5. O usuário preenche opcionalmente campos adicionais: dificuldade, limite de tempo e limite de memória, algoritmo de checagem.
6. O usuário envia o problema para publicação clicando no botão “Publicar”.
7. O sistema valida os dados inseridos:
    - Verifica se todos os campos obrigatórios foram preenchidos.
    - Confere se o formato das entradas está correto.
8. Se os dados forem válidos, o sistema armazena o problema no banco de dados.
9. O sistema exibe uma confirmação de que o problema foi publicado com sucesso.
10. O problema agora fica disponível para que outros usuários possam visualizá-lo e resolvê-lo.

[UC002] Submissão de solução

Descrição: permite que um usuário submeta uma solução para um problema de programação publicado no sistema.

Ator: Usuário

Prioridade	**X**	Essencial		Importante		Desejável

Entradas e pré-condições:

1. Pré-condições:
    - O usuário deve estar autenticado no sistema;
    - O usuário deve ter acesso à interface de submissão de soluções.
2. Entradas obrigatórias:
    - Seleção do problema a ser resolvido;
    - Código fonte da solução;
    - Linguagem de programação escolhida.
3. Entradas opcionais:
    - Comentário.

Saídas e pós-condições:

1. Saídas:
    - Mensagem de resultado da submissão: aprovada, reprovada ou erro de execução;
    - Relatório de testes (opcional) detalhando falhas ou sucesso.
2. Pós-condições
    - A solução é registrada no histórico do usuário.
    - Usuários podem curtir a solução para torná-la mais relevante.

Fluxo de eventos principal:

1. O usuário acessa a interface de submissão de soluções.
2. O usuário seleciona o problema que deseja resolver.
3. O usuário insere ou faz upload do código da solução.
4. O usuário escolhe a linguagem de programação utilizada.
5. O usuário envia a solução clicando em “Submeter”.
6. O sistema valida o envio:
    - Confere se todos os campos obrigatórios foram preenchidos.
7. A solução é registrada no histórico do usuário.

Fluxo de eventos secundários:

Fluxo secundário [001]:

1. Após a validação da solução enviada, se o problema possuir algoritmo de checagem, o sistema executa a solução contra os casos de teste do problema.
2. O sistema retorna se o problema foi capaz de ser resolvido ou não dentre os limites estabelecidos, e retorna uma exceção associada se não.
3. A solução é registrada no histórico do usuário.

[UC003] Criação de salas

Descrição: permite que um usuário crie salas no sistema, onde outros usuários podem entrar para interagir, resolver problemas de programação em conjunto ou participar de desafios coletivos.

Ator: Usuário

Prioridade	**X**	Essencial		Importante		Desejável

Entradas e pré-condições:

1. Pré-condições:
    - O usuário deve estar autenticado no sistema;
    - O usuário deve ter acesso à interface de criação de salas.
2. Entradas obrigatórias:
    - Tipo de sala (pública ou privada).
3. Entradas opcionais:
    - Senha de acesso (caso a sala seja privada);
    - Limite de participantes;
    - Chat habilitado.

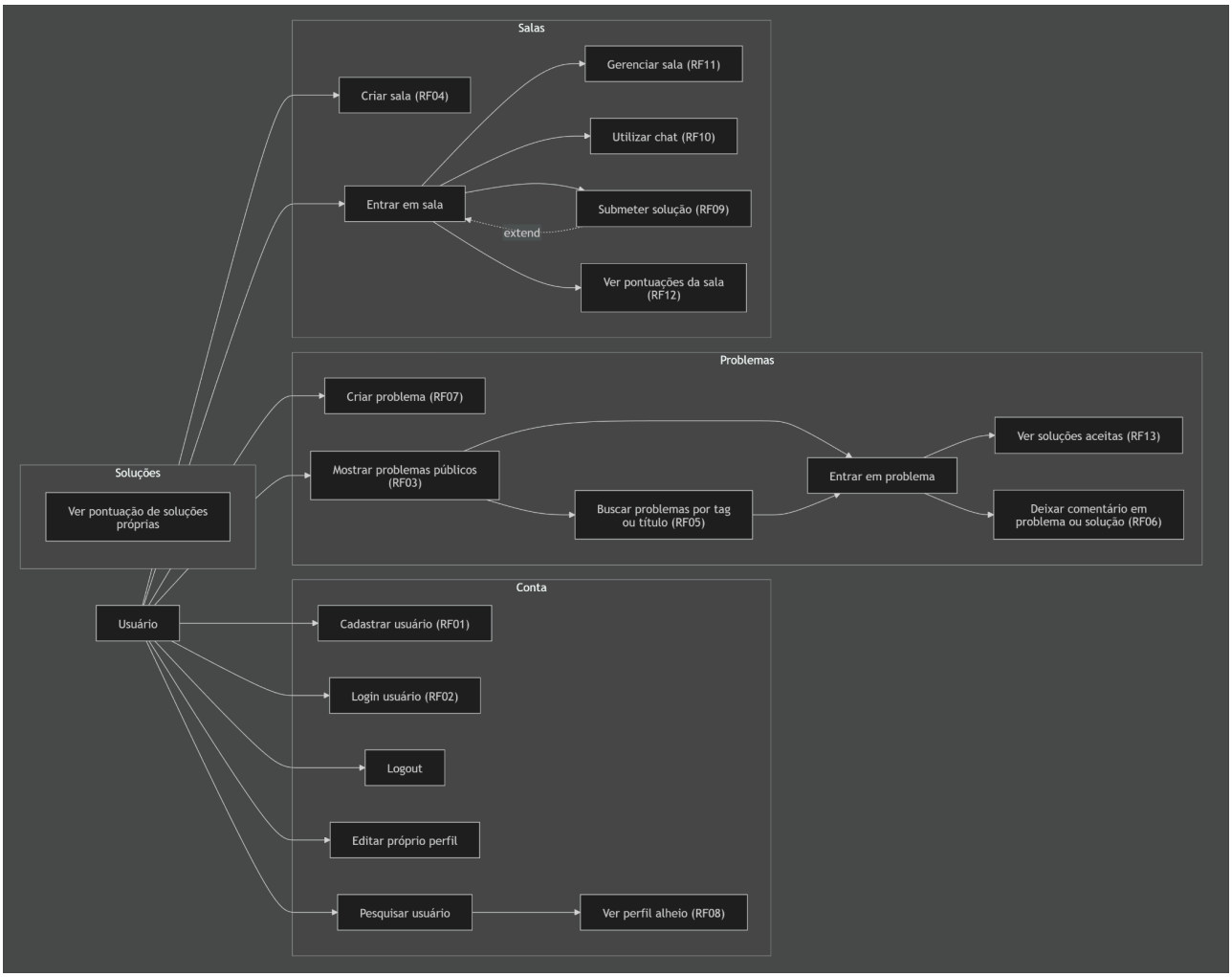
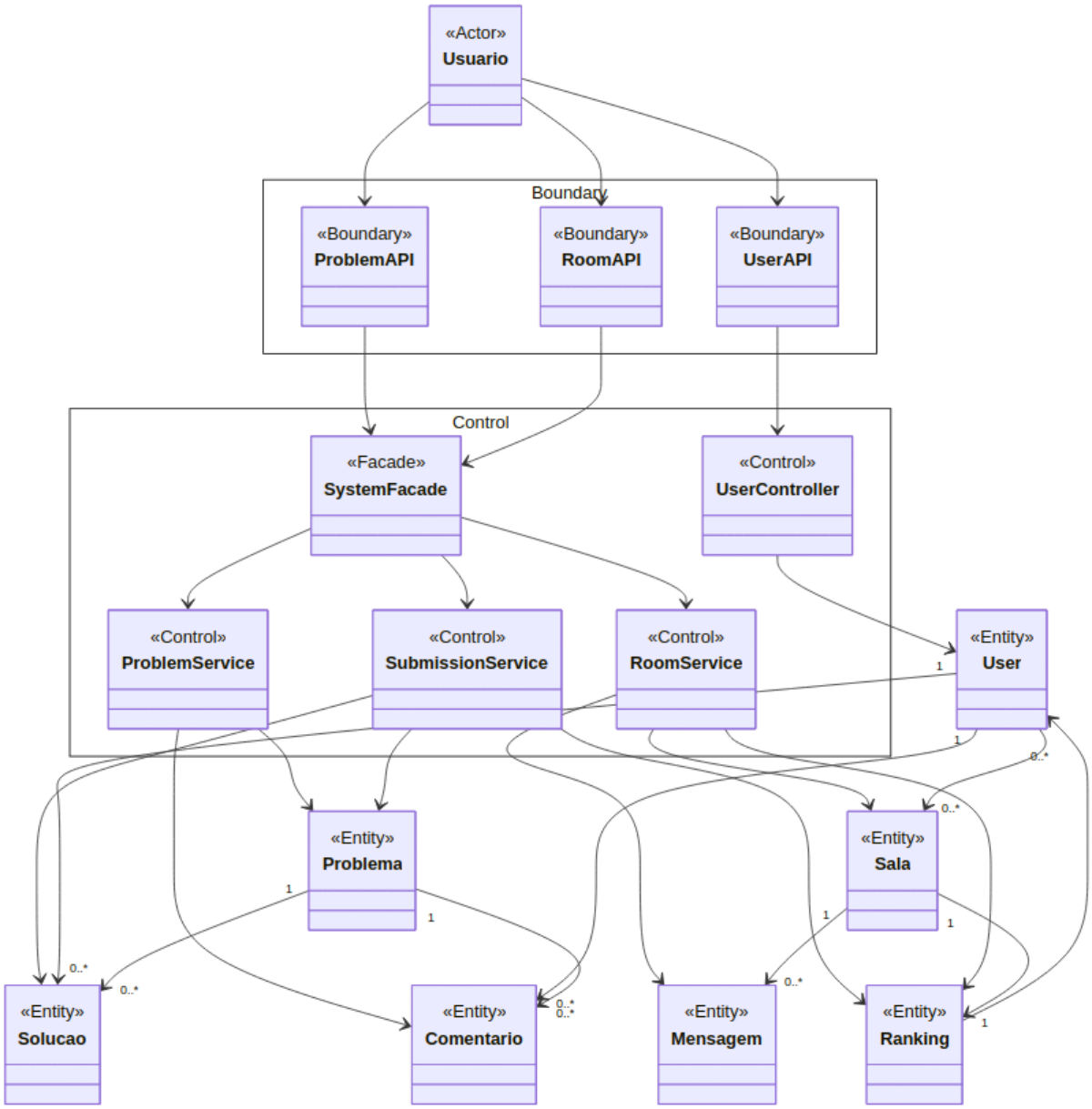
Saídas e pós-condições:

1. Saídas:
    - Confirmação de criação de sala exibida ao usuário;
    - Mensagem de erro detalhando problemas em caso de falha;
    - Código da sala para compartilhamento.
2. Pós-condições:
    - A sala é registrada no banco de dados do sistema;
    - A sala fica disponível para visualização por outros usuários (se pública) ou acessível; apenas mediante senha (se privada);
    - O usuário que criou a sala é automaticamente o administrador/moderador da sala;
    - Em caso de erro, a sala **não** é criada, garantindo consistência do sistema.

Fluxo de eventos principal:

1. O usuário acessa a interface de criação de salas.
2. O usuário escolhe o tipo (pública ou privada),.
3. O usuário preenche, se desejar, a senha, limite de participantes e chat habilitado.
4. O usuário clica em “Criar Sala”.
5. O sistema valida os dados:
    - Verifica se todos os campos obrigatórios foram preenchidos.
    - Confere se não existe outra sala com o mesmo nome.
6. O sistema registra a sala no banco de dados.
7. O sistema exibe a confirmação de criação da sala com o código.
8. A sala passa a estar disponível conforme seu tipo (pública ou privada).

6.1. Diagrama Casos de Uso



7. ANÁLISE DE CASOS DE USO (DIAGRAMA DE CLASSES DE ANÁLISE)